

Module 2

Transfer Learning, Séquences & XAI

Fine-tuning, LSTM/GRU, interprétabilité Grad-CAM / SHAP / LIME / IG

De la réutilisation à l'explication

ImageNet → tâches cibles — mémoire à long terme — audit de modèles



Transfer

Fine-tuning

h_t

LSTM / GRU

Portes, BPTT

$\nabla_x f$

Grad-CAM

Saliency

ϕ_i

SHAP / LIME

Attribution

Bassem Ben Hamed

bassem.benhamed@enetcom.usf.tn

ENETCOM — Université de Sfax

Présentation 02 — Module 2

1. Transfer Learning : principe & workflow
2. Modèles récurrents : RNN, LSTM, GRU

3. Explainable AI (XAI) — pourquoi & comment
4. Synthèse & références

Transfer Learning : principe & workflow

Transfer Learning — formulation mathématique

Cadre formel

Soit un modèle source f_{θ_S} entraîné sur $(\mathcal{X}_S, \mathcal{Y}_S) \sim \mathcal{D}_S$ (ex. ImageNet, 1.28M images, 1000 classes). On cherche un modèle cible f_{θ_T} pour $(\mathcal{X}_T, \mathcal{Y}_T) \sim \mathcal{D}_T$ avec $|\mathcal{D}_T| \ll |\mathcal{D}_S|$.

$$\theta_T = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}_T} [\mathcal{L}(f_{\theta}(x), y)] \quad \text{avec init. } \theta_{\text{init}} = \theta_S.$$

Hypothèse de transférabilité

Les **features bas/intermédiaires** (arêtes, textures, formes) sont *universelles* en vision et réutilisables. Les *features haut-niveau* sont spécifiques à la tâche source $\rightarrow$ à remplacer.

Quand l'utiliser ?

- Peu de données (10^2 – 10^4 images)
- Domaine proche d'ImageNet (photos naturelles)
- Budget GPU limité — entraînement $\times 10$ à $\times 100$ plus rapide

Transfer Learning — deux stratégies

Feature Extraction



`requires_grad=False` sur tout le backbone

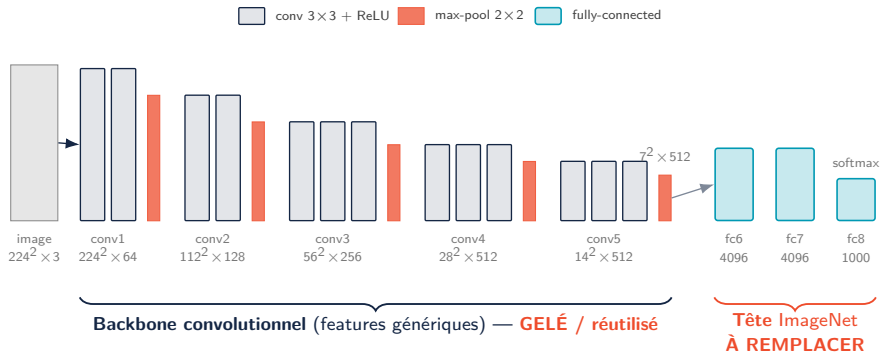
Fine-Tuning complet



LR différentiel : backbone 10^{-5} , tête 10^{-3}

Stratégie	Quand ?	Risques
Feature extraction	Données très limitées, domaine proche d'ImageNet, GPU faible	Capacité d'adaptation réduite
Fine-tuning partiel	Quelques milliers d'exemples, domaine voisin	Overfitting des dernières couches
Fine-tuning complet	$> 10^4$ exemples, domaine éloigné (médical, satellite)	Catastrophic forgetting si LR trop grand

Anatomie d'un backbone pré-entraîné — l'exemple VGG16



Comment lire ce schéma

En avançant, la **résolution spatiale** chute ($224^2 \rightarrow 7^2$) pendant que la **profondeur** (canaux) grimpe ($3 \rightarrow 512$) : le réseau passe des « pixels » aux « concepts ».

Lien direct avec le transfer learning

VGG16 (Simonyan & Zisserman, 2014) : 13 conv + 3 FC, $\sim 138\text{M}$ params. On **gèle** le backbone et on **remplace fc8** (1000 classes ImageNet) par une tête adaptée à la tâche cible.

Workflow PyTorch — fine-tuning ResNet50

Recette canonique (4 étapes)

1. Charger un modèle pré-entraîné depuis
`torchvision.models`
2. Geler le backbone : `param.requires_grad = False`
3. Remplacer la tête finale par une couche adaptée à la nouvelle tâche
4. Entraîner avec un **learning rate différentiel**

⚠ Normalisation cohérente

Les modèles ImageNet attendent des entrées normalisées avec

$$\mu=(0.485, 0.456, 0.406), \sigma=(0.229, 0.224, 0.225)$$

⇒ utiliser ces mêmes statistiques côté cible.

```
from torchvision import models
from torchvision.models import ResNet50_Weights

model = models.resnet50(
    weights=ResNet50_Weights.DEFAULT)

# 1. Geler le backbone
for p in model.parameters():
    p.requires_grad = False

# 2. Remplacer la tete (1000 -> 2)
model.fc = nn.Linear(2048, 2)

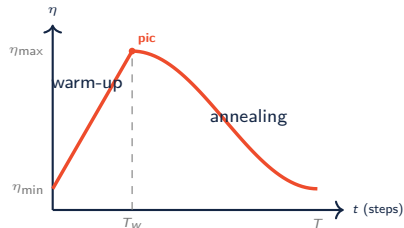
# 3. Optimiseur a LR differentiel
opt = optim.Adam([
    {"params": model.fc.parameters(),
     "lr": 1e-3},
    {"params": model.layer4.parameters(),
     "lr": 1e-5},
], weight_decay=1e-4)
```

OneCycleLR — monter *puis* redescendre le learning rate

Idée : un seul grand cycle (Smith, 2018)

Au lieu d'un LR fixe, on **monte** le LR puis on le **redescend** sur toute la durée d'entraînement, en 3 temps :

1. **Warm-up** ($\sim 30\%$) : η monte de $\eta_{\min}$ à $\eta_{\max}$. *Démarrage doux* $\Rightarrow$ BatchNorm & Adam se stabilisent.
2. **Pic** à $\eta_{\max}$: grand LR = *régularisateur*; le modèle explore large et évite les minima « pointus ».
3. **Annealing** (cosinus) : $\eta \rightarrow \eta_{\min}$; on se *pose finement* dans un bon minimum.



En pratique :

```
OneCycleLR(opt, max_lr=1e-3,  
total_steps=T, pct_start=0.3)
```

Lecture terme à terme

$$\eta(t) = \begin{cases} \eta_{\min} + (\eta_{\max} - \eta_{\min}) \frac{t}{T_w}, & t \leq T_w \\ \eta_{\min} + (\eta_{\max} - \eta_{\min}) \frac{1 + \cos\left(\pi \frac{t - T_w}{T - T_w}\right)}{2}, & t > T_w \end{cases}$$

$\eta_{\max}$: LR au pic — **le seul réglage vraiment clé** (`max_lr`), repéré par un *LR range test*.

$\eta_{\min} = \eta_{\max}/25$: LR de départ (`div_factor`).

$T_w = \text{pct_start} \cdot T \approx 0.3 T$: fin du warm-up.

T : nombre total de steps; t : step courant.

Cas d'usage — domaines médical & industriel

Domaine	Modèle source	Stratégie	Métriques cibles
Radiologie thoracique	ResNet50 ImageNet	Fine-tuning des 2 derniers blocs	AUC-ROC, sensibilité
Rétinopathie diabétique	EfficientNet-B4 ImageNet	Fine-tuning complet (30k images)	Kappa, F1 macro
Défauts industriels	EfficientNet-B0 ou ResNet18	Feature extraction (peu de défauts)	Précision @ rappel fixé
Imagerie satellite	ResNet50 ImageNet	Fine-tuning complet (domaine éloigné)	mIoU, accuracy par classe
Dermatoscopie ISIC	EfficientNet pré-entraîné	TF + augmentation forte (ColorJitter , RandAugment)	Balanced accuracy

Règle empirique : plus le domaine cible est éloigné d'ImageNet, plus il faut fine-tuner profondément.

Modèles récurrents : RNN, LSTM, GRU

Modèles de séquences — formulation

Cadre

Soit une séquence d'entrée $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ avec $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$. Un réseau récurrent maintient un **état caché** $\mathbf{h}_t \in \mathbb{R}^{d_h}$ qui synthétise l'historique :

$$\mathbf{h}_t = \phi_{\theta}(\mathbf{h}_{t-1}, \mathbf{x}_t), \quad \mathbf{y}_t = g_{\theta}(\mathbf{h}_t).$$

Les paramètres θ sont *partagés à travers le temps* (réutilisés à chaque pas).

Tâches typiques en séries temporelles :

- Prédiction : $\hat{\mathbf{x}}_{T+h}$ à partir de $(\mathbf{x}_1, \dots, \mathbf{x}_T)$
- Détection d'anomalies : score de surprise sur l'erreur de reconstruction
- Classification de séquences : geste, état machine, qualité du processus
- Maintenance prédictive : RUL (Remaining Useful Life)

Pourquoi pas un MLP ou un CNN 1D ?

- MLP $\rightarrow$ taille d'entrée fixée, pas de mémoire longue
- CNN 1D $\rightarrow$ champ réceptif borné par la profondeur
- RNN $\rightarrow$ **contexte théoriquement infini** (en pratique : limité par le vanishing gradient)

RNN vanille — équations, BPTT & vanishing gradient

Équations du RNN (Elman, 1990)

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Paramètres : $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_{in}}$, $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{W}_{hy} \in \mathbb{R}^{d_{out} \times d_h}$.

BPTT (Backpropagation Through Time)

Le gradient se propage à travers tous les pas temporels :

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \prod_{k=t+1}^T \mathbf{w}_{hh}^\top \text{diag}(\tanh'(z_k))$$

⚠ Vanishing / Exploding gradient

Si $\|\mathbf{W}_{hh}^\top \text{diag}(\tanh'(\cdot))\| < 1$, le produit s'écrase exponentiellement avec T :

$$\left\| \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t} \right\| \sim \rho^{T-t}, \quad \rho < 1.$$

⇒ **Impossibilité d'apprendre** les dépendances longues ($T > 50$).

Remède architectural : **LSTM, GRU**.

Remède numérique :

`torch.nn.utils.clip_grad_norm_` (clipping).

LSTM — vue d'ensemble du bloc

Idée centrale (Hochreiter & Schmidhuber, 1997)

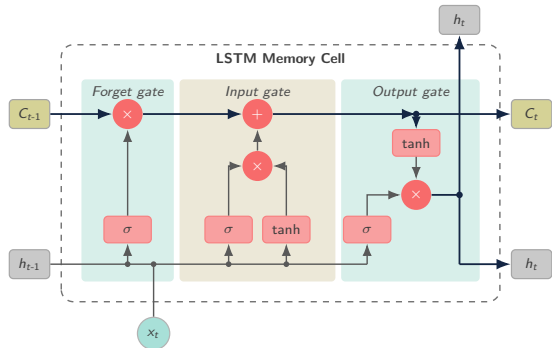
Introduire un **état cellulaire** c_t séparé de h_t , modifié par des *portes* (gates) qui contrôlent finement l'écriture, la lecture et l'oubli de l'information.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Trois portes sigmoïdes $\in (0, 1)^{d_h}$:

- f_t : oubli (*forget*)
- i_t : entrée (*input*)
- o_t : sortie (*output*)

Et un candidat de mise à jour $\tilde{c}_t \in (-1, 1)^{d_h}$ (tanh).



Bloc LSTM : 3 portes σ (forget/input/output) + 1 candidat tanh, rail d'état C_t

LSTM — équations détaillées des 4 portes

Notations

Soient $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$, $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$. Concaténation : $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h+d_{in}}$. Chaque porte/candidat utilise une matrice $\mathbf{W}_\star \in \mathbb{R}^{d_h \times (d_h+d_{in})}$ et un biais $\mathbf{b}_\star \in \mathbb{R}^{d_h}$.

Porte d'oubli (forget) — que jeter ?

$$\mathbf{f}_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$f_t^{(k)} \approx 0$: on efface le k -ième neurone de $\mathbf{c}_{t-1}$.

$f_t^{(k)} \approx 1$: on garde l'information.

Porte d'entrée (input) — que retenir ?

$$\mathbf{i}_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$\mathbf{i}_t$ filtre les nouvelles informations candidates $\tilde{\mathbf{c}}_t$.

Mise à jour de l'état cellulaire

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{oubli sélectif}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{ajout sélectif}}$$

Le **constant error carousel** : si $\mathbf{f}_t = 1$ et $\mathbf{i}_t = 0$, alors $\mathbf{c}_t = \mathbf{c}_{t-1}$ et le gradient passe à 1 (pas de vanishing).

Porte de sortie (output) — que révéler ?

$$\mathbf{o}_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

$\mathbf{h}_t$ est la partie de $\mathbf{c}_t$ exposée aux couches suivantes.

LSTM — comptage des paramètres

Décompte par bloc LSTM

Chaque porte (f , i , o) et le candidat (c) ont une matrice ($d_h \times (d_h + d_{in})$) et un biais (d_h). Soit **4 transformations affines** :

$$\#\theta_{\text{LSTM}} = 4 \cdot [d_h(d_h + d_{in}) + d_h] = 4 d_h (d_h + d_{in} + 1).$$

Exemple — $d_{in}=3$ (capteur 3 axes), $d_h=64$

$$4 \cdot 64 \cdot (64 + 3 + 1) = 4 \cdot 64 \cdot 68 = 17\,408 \text{ paramètres.}$$

Sur 2 couches empilées : $\sim 51\,200$ (la 2^e couche reçoit $d_h=64$ en entrée).

Initialisation du biais d'oubli

`nn.LSTM` initialise les biais à 0 *sauf* b_f qui peut être pré-réglé à 1 pour favoriser la mémorisation au démarrage (Jozefowicz et al., 2015).

Composant	Poids	Biais
Forget f_t	W_f	b_f
Input i_t	W_i	b_i
Candidat $\tilde{c}_t$	W_c	b_c
Output o_t	W_o	b_o
Total	$4 d_h (d_h + d_{in})$	$4 d_h$

PyTorch concatène les 4 poids dans un seul tenseur `weight_ih_l0` (forme $4d_h \times d_{in}$).

LSTM — exemple numérique pas à pas

Configuration jouet : $d_{in}=1$, $d_h=1$, $x_t = 1.0$, $h_{t-1} = 0.5$, $c_{t-1} = 0.2$

Poids fixés : $W_f=W_i=W_c=W_o=[1.0, 1.0]$ (sur $[h_{t-1}, x_t]$), tous biais à 0. σ : sigmoïde.

Étape 1 — calcul des portes

$$z = 1.0 \cdot 0.5 + 1.0 \cdot 1.0 + 0 = 1.5$$

$$f_t = \sigma(1.5) = 0.818$$

$$i_t = \sigma(1.5) = 0.818$$

$$o_t = \sigma(1.5) = 0.818$$

$$\tilde{c}_t = \tanh(1.5) = 0.905$$

Étape 2 — état & sortie

$$\begin{aligned} c_t &= f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \\ &= 0.818 \cdot 0.2 + 0.818 \cdot 0.905 \\ &= 0.164 + 0.740 = 0.904 \end{aligned}$$

$$\begin{aligned} h_t &= o_t \cdot \tanh(c_t) \\ &= 0.818 \cdot \tanh(0.904) \\ &= 0.818 \cdot 0.718 = \mathbf{0.587} \end{aligned}$$

💡 Lecture

Avec $f_t=0.82$, on conserve $\sim 82\%$ de la mémoire passée et on en ajoute $\sim 82\%$ du nouveau candidat $\tilde{c}_t$. C'est le **réglage dynamique** appris par les portes : selon le contexte (h_{t-1}, x_t) , le réseau choisit d'oublier plus ou moins.

L'entraînement ajuste W_f, W_i, W_c, W_o via BPTT.

GRU — alternative légère au LSTM

Gated Recurrent Unit (Cho et al., 2014)

Fusionne c_t et h_t , et regroupe les portes :

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{reset})$$

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{update})$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

⇒ **3 transformations affines** au lieu de 4.

Aspect	LSTM	GRU
# transformations	4	3
# paramètres	$4d_h(d_h + d_{in} + 1)$	$3d_h(d_h + d_{in} + 1)$
États maintenus	c_t, h_t	h_t
Mémoire longue	Meilleure (carousel)	Bonne
Vitesse / mémoire	Plus lourd	Plus rapide

Empiriquement, GRU = LSTM à $\pm 1\%$ de performance pour de nombreuses tâches ; choisir GRU si latence/mémoire critique.

RNN bidirectionnel & bonnes pratiques d'entraînement

Bidirectionnel : passé + futur

Deux RNN parallèles parcourent la séquence dans les deux sens :

$$h_t = [\vec{h}_t, \overleftarrow{h}_t] \in \mathbb{R}^{2d_h}$$

Doublé le nombre de paramètres mais accède au contexte futur (ex. fenêtre glissante hors-ligne, post-traitement).

Non applicable au *forecasting en ligne* (le futur n'est pas disponible).

Gradient clipping

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|_2}\right), \quad \tau \approx 1.0$$

Évite l'explosion lors des longues séquences.

```
lstm = nn.LSTM(
    input_size=3, hidden_size=64,
    num_layers=2, bidirectional=True,
    dropout=0.2, batch_first=True)
```

```
x = torch.randn(32, 100, 3)  # (B,T,F)
y, (h_n, c_n) = lstm(x)
# y.shape : (32, 100, 128)  # 2*64
```

```
# Sequences de longueurs variables :
packed = nn.utils.rnn.pack_padded_sequence(
    x, lengths, batch_first=True,
    enforce_sorted=False)
out, _ = lstm(packed)
out, _ = nn.utils.rnn.pad_packed_sequence(
    out, batch_first=True)
```

```
# Clipping pendant l'entraînement :
torch.nn.utils.clip_grad_norm_(
    lstm.parameters(), max_norm=1.0)
```

Cas d'usage — séries temporelles industrielles

Tâche	Entrée	Architecture	Métriques
Prévision conso. énergie	Historique 24h, $T=96$ pas, 1 feature	LSTM 2 couches $d_h=64$	MAE, RMSE, MAPE
Détection d'anomalies IoT	Capteurs 6 axes, fenêtre glissante	BiLSTM + seuil reconstruction	AUC-ROC, F1
Maintenance prédictive	Vibrations + températures (RUL)	LSTM + tête régression	MAE en cycles
Qualité de procédé	Mesures de production multi-canaux	GRU + classification	F1 macro
Trafic réseau	Volumes par minute, plusieurs services	LSTM multi-step	sMAPE, MAE

💡 Pipeline type

Normalisation par fenêtre (z-score sur passé glissant) → BiLSTM ou GRU → tête **Linear** → loss MSE / CE selon la tâche.

Explainable AI (XAI) — pourquoi & comment

Pourquoi expliquer un modèle ?

- **Confiance** : un radiologue ne signe pas un diagnostic d'IA *boîte noire*
- **Conformité** (IA Act, FDA, RGPD art. 22) : droit à l'explication
- **Débogage** : détecter les *shortcuts* (artefact d'arrière-plan, biais)
- **Sécurité** : caractériser les modes de défaillance

Famille	Principe	Exemples	Coût
Gradient-based	$\nabla_x f$ et variantes spatialisées	Saliency, Grad-CAM, IG	Faible (1 backward)
Perturbation	Masquer / changer l'entrée, mesurer l'effet	Occlusion, RISE, LIME	Élevé (N forward)
Approximation locale	Modèle linéaire interprétable autour de x	LIME	Moyen
Théorie des jeux	Valeurs de Shapley sur features	SHAP	Élevé
Attention	Cartes d'attention multi-couches	Attention Rollout (ViT)	Faible

XAI — vocabulaire essentiel & cas d'usage réels

Trois distinctions pour s'y retrouver

- **Locale** : « pourquoi *cette* prédiction ? » (Grad-CAM, LIME, IG) – **globale** : « comment le modèle raisonne *en général* ? » (SHAP agrégé sur tout le jeu de données).
- **Post-hoc** : on explique un réseau *déjà entraîné*, sans le modifier – **interprétable par conception** : le modèle est lisible en lui-même (régression linéaire, petit arbre).
- **Agnostique** : n'utilise que les entrées/sorties, aucune connaissance interne (LIME, KernelSHAP) – **spécifique** : exploite les gradients ou l'architecture (Grad-CAM, IG, Rollout).

Secteur	Question métier concrète	Méthode typique
Santé	Le réseau regarde-t-il bien la <i>lésion</i> , et non l'étiquette de l'hôpital sur la radio ?	Grad-CAM sur radio / IRM
Banque / crédit	Pourquoi ce dossier est-il refusé ? (explication exigée par le régulateur)	SHAP <i>waterfall</i> par client
Industrie	Quel défaut de surface a déclenché le rejet de cette pièce ?	Grad-CAM / Occlusion
Assurance, RH	Quels mots du dossier ou variables ont pesé dans ce score ?	LIME texte, SHAP tabulaire
Véhicule autonome	Qu'a « vu » le système juste avant la décision de freinage ?	IG, attention (audit d'incident)

XAI en images — que « voit-on » concrètement ?

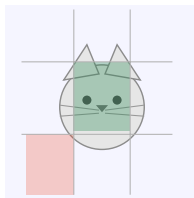


Image d'entrée

prédit : chat (0,93)

Grad-CAM

où : zone grossière



LIME

quelles régions (+ / -)



Integrated Grad.

quels pixels (fins)

💡 Comment lire ces sorties

Même image, trois *granularités* de réponse : **Grad-CAM** surligne une *zone* (« où »), **LIME** des *régions*/superpixels (en **vert** ce qui *soutient* la classe, en orange ce qui s'y *oppose*), **Integrated Gradients** (et SHAP) des *pixels* fins. **Règle d'or** : une bonne explication se concentre sur le **sujet**, pas sur le fond.

XAI en images — sur quoi chaque méthode travaille-t-elle ?

Méthode	Travaille sur...	Comment, concrètement	Carte obtenue
Saliency	les pixels bruts	1 backward : $ \partial f_c / \partial x_i $ pour chaque pixel	pixels (fine, bruitée)
Grad-CAM	les cartes de features de la <i>dernière conv</i>	pondère chaque carte par son gradient moyen, somme + ReLU, ré-agrandissement ; ne perturbe jamais l'entrée	zones grossières ($\sim 7 \times 7$)
Integrated Gradients	les pixels bruts	cumule les gradients le long d'un <i>fondus</i> baseline (image noire) $\rightarrow$ image	pixels (fine)
SHAP Gradient / Deep	les pixels bruts	gradients par rapport à des baselines tirées au hasard — cousins directs d'IG	pixels (fine)
SHAP Kernel / Partition	des superpixels	valeurs de Shapley sur des coalitions de segments (2^n coalitions $\Rightarrow$ impossible au pixel près)	régions
LIME	des superpixels	segmente l'image (SLIC, quickshift), active/désactive des zones au hasard, régression linéaire locale pondérée	régions ($\pm$)



Deux logiques seulement

Perturber des superpixels et observer le score (LIME, Kernel/PartitionSHAP : model-agnostic, N forwards), ou **suivre les gradients** — par rapport aux *pixels* (Saliency, IG, Gradient/DeepSHAP) ou aux *cartes internes* du CNN (Grad-CAM, le seul à ne pas travailler sur l'entrée). Chaque brique (carte de features, baseline, superpixel) est détaillée dans les slides qui suivent.

Grad-CAM — c'est quoi une « carte d'activation » A^k ?

Une carte = la sortie d'un filtre

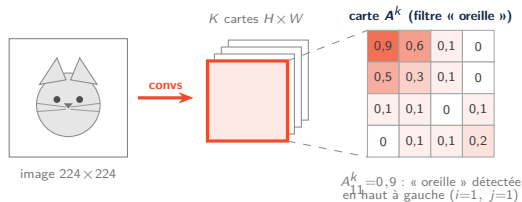
Une couche de convolution à K filtres ne sort pas une image mais un **tenseur** $K \times H \times W$:

- le filtre k glisse sur l'image et détecte **un motif** appris (oreille, fourrure, herbe...);
- sa réponse forme une grille $H \times W$ de nombres : la **carte** A^k ;
- la case A^k_{ij} est **grande** si le motif est présent à la position (i, j) , ≈ 0 sinon.

Une carte dit donc *ce qui* a été détecté (le filtre) et *où* (la position) : exactement l'information dont Grad-CAM a besoin.

Ordres de grandeur (entrée 224×224)

ResNet-18, **layer4** : $K=512$ cartes 7×7 . Chaque case « couvre » un patch d'environ 32×32 px de l'image d'origine — d'où la heatmap *grossière*.



Grad-CAM — « où le réseau a-t-il regardé ? »

Idée (Selvaraju et al., 2017)

On prend les cartes d'activation $\mathbf{A}^k$ de la *dernière couche conv*, on pèse chacune par son **importance** pour la classe c , puis on les superpose en une carte de chaleur :

$$\underbrace{\alpha_c^k = \frac{1}{H W} \sum_{i,j} \frac{\partial f_c}{\partial A_{ij}^k}}_{\text{poids de la carte } k} \quad \underbrace{L_c = \text{ReLU}\left(\sum_k \alpha_c^k \mathbf{A}^k\right)}_{\text{carte de chaleur finale}}$$

Lecture terme à terme

- f_c : score (logit, *avant softmax*) de la classe expliquée c .
- $\mathbf{A}^k$: k -ième *carte d'activation* — ce que le filtre k détecte, et où.
- $\partial f_c / \partial A_{ij}^k$: *sensibilité* — A_{ij}^k augmente $\Rightarrow f_c$ monte (> 0) ou baisse (< 0) ?
- α_c^k : moyenne (GAP) $\Rightarrow$ « cette carte compte-t-elle pour c ? ».
- ReLU : on ne garde que ce qui **pousse vers** c (le négatif $\rightarrow 0$).

À retenir

Carte de chaleur **grossière** ($\sim 7 \times 7$) ré-agrandie sur l'image : elle montre les *zones décisives* (idéale en imagerie médicale / défauts).

Variantes utiles

Grad-CAM++ (objets multiples), **HiResCAM** (carte plus fine), **EigenCAM** (sans gradient).

Grad-CAM — l'exemple complet, calculé à la main

Réseau jouet : $K=2$ cartes 2×2 ; classe expliquée $c = \text{« chat »}$

Une couche linéaire (poids w_{ij}^k) sur les 8 activations aplaties donne le logit $f_c = \sum_k \sum_{i,j} w_{ij}^k A_{ij}^k$, avec :

$$\mathbf{A}^1 = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \text{ « oreille »}, \quad \mathbf{A}^2 = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \text{ « herbe »}, \quad \mathbf{w}^1 = \begin{pmatrix} 0,8 & 0,4 \\ 0,2 & 0,2 \end{pmatrix}, \quad \mathbf{w}^2 = \begin{pmatrix} -0,2 & 0 \\ 0 & -0,6 \end{pmatrix}$$

Déroulé du calcul en 3 étapes

1. **Backward** — dériver une somme = garder le coefficient de l'activation :

$$\frac{\partial f_c}{\partial A_{ij}^k} = w_{ij}^k \Rightarrow \frac{\partial f_c}{\partial \mathbf{A}^1} = \begin{pmatrix} 0,8 & 0,4 \\ 0,2 & 0,2 \end{pmatrix}, \quad \frac{\partial f_c}{\partial \mathbf{A}^2} = \begin{pmatrix} -0,2 & 0 \\ 0 & -0,6 \end{pmatrix}$$

2. **Moyenne spatiale (GAP)** — $H=W=2$, donc 4 cases :

$$\alpha_c^1 = \frac{0,8+0,4+0,2+0,2}{4} = 0,4 \quad \alpha_c^2 = \frac{-0,2+0+0-0,6}{4} = -0,2$$

3. **Combinaison + ReLU** — case par case :

$$L_c = \text{ReLU}(0,4 \mathbf{A}^1 - 0,2 \mathbf{A}^2) = \text{ReLU}\left(\begin{pmatrix} 0,4 & 0 \\ 0 & -0,2 \end{pmatrix}\right) = \begin{pmatrix} 0,4 & 0 \\ 0 & 0 \end{pmatrix}$$

Comment interpréter ?

- Signe : $+0,8 \Rightarrow$ l'« oreille » fait **monter** f_c (preuve *pour*) ; $-0,6 \Rightarrow$ l'« herbe » joue *contre*.
- $\alpha_c^1 = 0,4 > 0$: la carte « oreille » est conservée ;
 $\alpha_c^2 = -0,2 < 0$: le ReLU **efface** sa contribution.
- La chaleur finale se concentre en haut à gauche = là où l'oreille a été détectée.

Et dans un vrai réseau ?

Entre $\mathbf{A}^k$ et f_c (pooling, densés...), `score.backward()` applique la **règle de la chaîne**. Enfin L_c est ré-agrandie sur l'image : la heatmap.

Grad-CAM — algorithme & code PyTorch

Algorithme (5 étapes)

1. *Forward* jusqu'au logit cible f_c
2. *Hook* sur la couche cible pour capturer A^k
3. *Backward* de f_c pour obtenir $\partial f_c / \partial A^k$
4. GAP sur l'axe spatial $\rightarrow \alpha_c^k$
5. Combinaison linéaire + ReLU + *upsample* à la taille de l'image

Couche cible

Pour ResNet : dernière convolution (`layer4[-1]`).

Pour un ViT : reshape de la dernière map d'attention.

```
def grad_cam(model, x, target_class, target_layer):
    feats, grads = [], []
    h1 = target_layer.register_forward_hook(
        lambda m, i, o: feats.append(o))
    h2 = target_layer.register_full_backward_hook(
        lambda m, gi, go: grads.append(go[0]))

    logits = model(x)                # (1, C)
    score = logits[0, target_class]
    model.zero_grad(); score.backward()
    h1.remove(); h2.remove()

    A = feats[0][0]                  # (K, H, W)
    G = grads[0][0]                  # (K, H, W)
    alpha = G.mean(dim=(1,2))        # (K,)
    cam = torch.relu(
        (alpha[:,None,None] * A).sum(0))
    return cam / (cam.max()+1e-8)
```

Saliency — la plus simple : le gradient par rapport aux pixels

Idée (Simonyan et al., 2014) : « quels pixels feraient bouger le score ? »

Le réseau entier est vu comme une fonction F qui mange *tous* les pixels et rend **un seul nombre** : le score de la classe expliquée (le f_c de Grad-CAM). La carte de saliency est son gradient *brut* :

$$F : \underbrace{x \in \mathbb{R}^{3 \times 224 \times 224}}_{\text{tous les pixels}} \mapsto \underbrace{F(x) \in \mathbb{R}}_{\text{score de la classe}}$$

$$S_i = \left| \frac{\partial F}{\partial x_i} \right| \quad (\text{max sur les 3 canaux RGB})$$

Exemple à la main : la règle de la chaîne

Réseau jouet : 2 pixels, 1 neurone caché, 1 sortie :

$$h = \text{ReLU}(2x_1 - x_2), \quad F = 3h$$

Entrée $x_1=0,8$, $x_2=0,2$: $h = \text{ReLU}(1,6 - 0,2) = 1,4$ et $F = 4,2$. On remonte couche par couche (ReLU actif $\Rightarrow$ sa dérivée vaut 1) :

$$\frac{\partial F}{\partial x_1} = \underbrace{\frac{\partial F}{\partial h}}_3 \cdot \underbrace{\frac{\partial h}{\partial x_1}}_{1 \cdot 2} = 6 \qquad \frac{\partial F}{\partial x_2} = 3 \cdot (1 \cdot (-1)) = -3$$

Carte de saliency : $S = (|+6|, |-3|) = (6, 3)$ — le pixel x_1 pèse deux fois plus. Le signe (avant $|\cdot|$) dit le sens : $+6$ pousse vers la classe, -3 la freine.

Saliency vs Grad-CAM

- Saliency dérive par rapport aux **pixels d'entrée** $\rightarrow$ carte *fine* (224×224) mais **bruitée**.
- Grad-CAM dérive par rapport aux **cartes de la dernière conv** $\rightarrow$ carte *grossière* ($\sim 7 \times 7$) mais nette.
- Même coût : **1 backward** (x.grad rempli d'un coup).

Limites $\Rightarrow$ méthodes suivantes

Bruit $\rightarrow$ moyenner sur des copies bruitées (SmoothGrad).
Saturation (gradient ≈ 0 sur pixel décisif) $\rightarrow$ IG.

Integrated Gradients — cumuler le gradient le long d'un chemin

Idée (Sundararajan et al., 2017)

Le gradient brut de la saliency *sature* : $\partial F / \partial x_i$ peut valoir 0 même pour un pixel décisif. Remède : partir d'une **baseline** neutre $\mathbf{x}'$ (image noire) et *cumuler* le gradient le long du chemin droit qui mène de $\mathbf{x}'$ à l'image $\mathbf{x}$:

$$\text{IG}_i(\mathbf{x}) = \underbrace{(x_i - x'_i)}_{\text{amplitude du pixel } i} \underbrace{\int_0^1 \frac{\partial F(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha}_{\text{gradient moyen le long du chemin}}.$$

Lecture terme à terme

- $\mathbf{x}'$: **baseline** = « absence d'information » (noir, gris, flou).
- $\alpha : 0 \rightarrow 1$: curseur qui glisse de la baseline ($\alpha=0$) à l'image ($\alpha=1$).
- $\partial F / \partial x_i$: sensibilité de la sortie au pixel i en chaque point du chemin.
- $(x_i - x'_i)$: de combien le pixel i a changé.

En pratique : moyenne de $m \in [20, 200]$ gradients ([n_steps](#)).

La bonne propriété (*complétude*)

$$\sum_i \text{IG}_i = F(\mathbf{x}) - F(\mathbf{x}')$$

Les attributions se **somment exactement** à l'écart de prédiction entre l'image et la baseline : rien n'est « perdu », chaque pixel reçoit sa juste part.

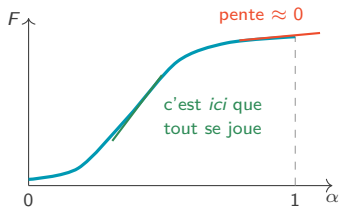
En pratique

Marche sur tout modèle différentiable (CNN, ViT, MLP) ; tester plusieurs baselines ; [captum.attr.IntegratedGradients](#). 27/41

Integrated Gradients — saturation, calcul pratique & lecture

Pourquoi le gradient seul « ment »

Score le long du chemin $F(x' + \alpha(x - x'))$: il monte, puis **sature**. À $\alpha=1$ (l'image réelle), la pente est quasi nulle alors que le pixel a été décisif.



Analogie : la pente au sommet est nulle, mais le dénivelé vient de tout le chemin $\Rightarrow$ on **cumule** la pente.

En machine : l'intégrale devient une moyenne

$$IG_i \approx (x_i - x'_i) \cdot \frac{1}{m} \sum_{k=1}^m \frac{\partial F\left(x' + \frac{k}{m}(x - x')\right)}{\partial x_i}$$

On fabrique m images interpolées entre le noir et l'image, un backward chacune, puis on moyenne les gradients. $m=50$ suffit souvent.

Vérifier avec la complétude (exemple chiffré)

$F(x)=0,93$ (chat) et $F(x')=0,01$ (image noire) $\Rightarrow$ la somme des attributions *doit* valoir 0,92. Captum renvoie l'écart résiduel (`convergence_delta`) : s'il est grand, **augmenter n_steps**.

Lire la carte pixel par pixel

$IG_i > 0$: le pixel a **poussé** vers la classe ; $IG_i < 0$: il l'a *freinée* ; ≈ 0 : ignoré. Une explication saine : les attributions du *sujet* dominant celles du fond.

SHAP — répartir *équitablement* la prédiction (Shapley)

Idée venue de la théorie des jeux (Shapley, 1953 ; Lundberg & Lee, 2017)

Les features sont des « joueurs » qui coopèrent pour produire la prédiction. Combien chacune mérite-t-elle ? On mesure son **apport marginal** — ce qu'elle ajoute quand on l'inclut — *moyenné sur tous les contextes possibles* :

$$\phi_i = \text{moyenne}_{S \subseteq \text{autres features}} \left[\underbrace{v(S \cup \{i\})}_{\text{score avec } i} - \underbrace{v(S)}_{\text{score sans } i} \right].$$

Lecture terme à terme

- S : un sous-ensemble de features déjà présentes (le « contexte »).
- $v(S)$: prédiction en n'utilisant *que* les features de S .
- $v(S \cup \{i\}) - v(S)$: **apport** de la feature i ajoutée à ce contexte.
- moyenne sur tous les S : on juge i dans *tous* les contextes $\Rightarrow$ contribution *juste*.

Propriété clé (*efficacité*)

$$\sum_i \phi_i = F(\mathbf{x}) - \mathbb{E}[F]$$

prédiction = moyenne du modèle + somme des contributions. Chaque ϕ_i dit de combien la feature i **pousse au-dessus / en dessous** de la moyenne.

En pratique (exact en $\mathcal{O}(2^n)$ $\rightarrow$ approx.)

TreeSHAP (arbres : exact & rapide) ; **DeepSHAP/GradientSHAP** (réseaux) ; **KernelSHAP** (boîte noire). Sur **image** : Kernel/Partition $\rightarrow$ *superpixels* (comme LIME) ; Gradient/Deep $\rightarrow$ *pixels* (cousins d'IG). Visus : *summary*, *waterfall*.

SHAP — exemple chiffré : accorder un crédit

Deux features : **revenu** (élevé) et **dette** (élevée). $v(S)$ = score d'acceptation prédit en ne connaissant que les features de S .

Les 4 coalitions possibles

$v(\emptyset)$ = moyenne du modèle $\mathbb{E}[F]$	0,50
$v(\{\text{revenu}\})$	0,80
$v(\{\text{dette}\})$	0,30
$v(\{\text{revenu}, \text{dette}\}) = F(x)$	0,70

Apport marginal de chaque feature dans ses 2 contextes :

	revenu	dette
ajoutée seule (à $\emptyset$)	$0,80 - 0,50 = +0,30$	$0,30 - 0,50 = -0,20$
ajoutée en 2 ^e	$0,70 - 0,30 = +0,40$	$0,70 - 0,80 = -0,10$
ϕ = moyenne des 2	+0,35	-0,15

Vérification (propriété d'efficacité)

$$\phi_{\text{revenu}} + \phi_{\text{dette}} = +0,20 = F(x) - \mathbb{E}[F] = 0,70 - 0,50 \quad \checkmark$$

La phrase à donner au client (waterfall)

« Votre dossier part de la moyenne (50 %) ; votre **revenu** l'améliore de +35 points, votre **dette** le dégrade de -15 points $\Rightarrow$ score final 70 %. » C'est exactement ce que dessine le graphique *waterfall* de la librairie [shap](#).

Formule exacte pour n features

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \underbrace{\frac{|S|! (n - |S| - 1)!}{n!}}_{\text{poids de } S} [v(S \cup \{i\}) - v(S)]$$

Le poids = probabilité que i arrive juste *après* les features de S quand on les ajoute dans un ordre aléatoire ($n=2$: $\frac{1}{2}$ et $\frac{1}{2}$, comme ci-contre). 2^n coalitions $\Rightarrow$ exact impossible au-delà de ~ 20 features, d'où Tree/Deep/KernelSHAP.

LIME — une explication *locale* par modèle simple

Idée (Ribeiro et al., 2016)

On n'explique pas f partout, juste **autour de l'exemple x** . On ajuste un modèle *simple et lisible* g (régression linéaire) qui imite f dans ce voisinage :

$$\xi(x) = \arg \min_g \underbrace{\mathcal{L}(f, g, \pi_x)}_{g \text{ imite } f \text{ près de } x} + \underbrace{\Omega(g)}_{g \text{ reste simple}} .$$

Lecture terme à terme

- g : modèle *interprétable* (linéaire) dont on lira les coefficients.
- π_x : poids de **proximité** — les points proches de x comptent plus.
- $\mathcal{L}$: erreur d'imitation de f par g (pondérée par π_x).
- $\Omega(g)$: pénalité de *complexité* (peu de termes $\Rightarrow$ lisible).

Sur une image

Découper en *superpixels* (SLIC, quickshift) $\rightarrow$ masquer au hasard $\rightarrow$ régression pondérée $\rightarrow$ coefficient = importance $\pm$ de chaque zone.

Forces / Faiblesses

- + *model-agnostic* (vraie boîte noire), tout type de données.
- + Sortie facile à montrer à un expert métier.
 - *Instable* : varie selon l'échantillonnage aléatoire.
 - Coûteux : $N \in [10^3, 10^4]$ prédictions ; choix de σ arbitraire.

LIME — la recette pas à pas sur une image

Les 5 étapes

1. **Segmenter** l'image en superpixels (algorithme SLIC ou quickshift ; ex. 40 zones visuellement cohérentes).
2. **Perturber** : créer N copies en masquant des zones au hasard — chaque copie est codée $z \in \{0, 1\}^{40}$ ($1 = \text{zone visible}$).
3. **Prédire** : passer chaque copie dans le réseau f (d'où le coût : N forward).
4. **Pondérer** : les copies peu masquées, proches de l'originale, comptent plus (π_x).
5. **Régresser** : ajuster $g(z) = w_0 + \sum_j w_j z_j$; le coefficient w_j est l'importance ($\pm$) de la zone j .

Ce que « voit » la régression

Copie perturbée	$f(\text{« chat »})$
rien de masqué	0,93
tête masquée	0,21
fond masqué	0,90
tête + fond masqués	0,18

Masquer la **tête** effondre le score $\Rightarrow w_{\text{tête}}$ grand et positif (zone verte). Masquer le fond ne change presque rien $\Rightarrow w_{\text{fond}} \approx 0$ (zone neutre).

Réglages qui changent l'explication

Nombre d'échantillons N (1000–5000), taille des superpixels, largeur du noyau π_x : les modifier peut **changer les zones trouvées** $\rightarrow$ relancer 2–3 fois et vérifier la stabilité avant de conclure.

LIME — la régression calculée à la main

Version jouet : 2 zones seulement (**tête**, **fond**) ; modèle local $g(z) = w_0 + w_{\text{tête}} z_{\text{tête}} + w_{\text{fond}} z_{\text{fond}}$, avec $z_j=1$ si la zone j est visible.

Chaque copie perturbée donne une équation

$(z_{\text{tête}}, z_{\text{fond}})$	f	équation
(1, 1) rien de masqué	0,93	$w_0 + w_{\text{tête}} + w_{\text{fond}} = 0,93$
(0, 1) tête masquée	0,21	$w_0 + w_{\text{fond}} = 0,21$
(1, 0) fond masqué	0,90	$w_0 + w_{\text{tête}} = 0,90$
(0, 0) tout masqué	0,18	$w_0 = 0,18$

On résout par simples soustractions :

$$w_{\text{tête}} = 0,93 - 0,21 = \textcolor{red}{+0,72} \quad w_{\text{fond}} = 0,93 - 0,90 = \textcolor{red}{+0,03}$$

Cohérent : $0,90 - 0,18 = 0,72$ aussi. Vérification :

$$0,18 + 0,72 + 0,03 = 0,93 \quad \checkmark$$

Comment interpréter ?

- $w_{\text{tête}} = +0,72$: la zone **porte** la prédiction $\rightarrow$ affichée en **vert**.
- $w_{\text{fond}} = +0,03 \approx 0$: zone neutre, non colorée.
- $w_0 = 0,18$: score quand tout est masqué.

Et en vrai ?

~ 40 zones et $N \in [10^3, 10^4]$ copies *aléatoires* : plus d'équations que d'inconnues $\Rightarrow$ **moindres carrés pondérés** par π_x (les copies peu masquées comptent plus). Même principe, résolu par la librairie **lime**.

Idée (Abnar & Zuidema, 2020)

Un ViT découpe l'image en *patches* (+ un token [CLS] qui porte la décision). Chaque couche produit une matrice d'attention $\mathbf{A}^{(l)}$: « quel token regarde quel token ? ». On **compose** ces attentions couche après couche :

$$\tilde{\mathbf{A}}^{(l)} = \frac{1}{2}(\mathbf{A}^{(l)} + \mathbf{I}), \quad \mathbf{R} = \tilde{\mathbf{A}}^{(L)} \dots \tilde{\mathbf{A}}^{(1)}.$$

La ligne du token [CLS] dans $\mathbf{R}$ donne l'importance de chaque patch.

Lecture terme à terme

- $\mathbf{A}^{(l)}$: attention de la couche l (token $\times$ token).
- $+\mathbf{I}$ puis $\times \frac{1}{2}$: on **ajoute la connexion résiduelle** (un token se garde lui-même), puis on renormalise.
- produit $\mathbf{R}$: flux d'attention *cumulé* de l'entrée à la sortie.

Mieux encore

Transformer Attribution (Chefer, 2021) = rollout + gradient $\Rightarrow$ localisation plus précise. **Grad-CAM** marche aussi sur ViT.

Attention Rollout — exemple à la main (3 tokens, 2 couches)

ViT jouet : 3 tokens — [CLS] (porte la décision), P_1 (museau du chat), P_2 (fond). Chaque *ligne* d'une matrice d'attention dit « qui regarde qui » (somme = 1).

Déroulé du calcul

1. **Résiduel** : $\tilde{\mathbf{A}} = \frac{1}{2}(\mathbf{A} + \mathbf{I})$. Ex. ligne [CLS] de la couche 1 :
 $\mathbf{A}_{[\text{CLS}]}^{(1)} = (0,2 \ 0,6 \ 0,2) \Rightarrow \tilde{\mathbf{A}}_{[\text{CLS}]}^{(1)} = (0,6 \ 0,3 \ 0,1)$. Au complet :

$$\tilde{\mathbf{A}}^{(1)} = \begin{pmatrix} 0,6 & 0,3 & 0,1 \\ 0,1 & 0,8 & 0,1 \\ 0,1 & 0,1 & 0,8 \end{pmatrix} \quad \tilde{\mathbf{A}}_{[\text{CLS}]}^{(2)} = (0,6 \ 0,3 \ 0,1)$$

2. **Composer** : seule la ligne [CLS] de $\mathbf{R} = \tilde{\mathbf{A}}^{(2)}\tilde{\mathbf{A}}^{(1)}$ nous intéresse ; c'est un produit ligne $\times$ matrice. Vers P_1 :

$$R_{P_1} = \underbrace{0,6 \cdot 0,3}_{\text{via [CLS]}} + \underbrace{0,3 \cdot 0,8}_{\text{via } P_1} + \underbrace{0,1 \cdot 0,1}_{\text{via } P_2} = 0,18 + 0,24 + 0,01 = \mathbf{0,43}$$

De même : $R_{[\text{CLS}]} = 0,36 + 0,03 + 0,01 = 0,40$; $R_{P_2} = 0,06 + 0,03 + 0,08 = 0,17$.
Somme = 1 ✓

Comment interpréter ?

- Importance des patches : $P_1 = 0,43 \gg P_2 = 0,17 \Rightarrow$ la heatmap allume le **museau**.
- Chaque terme = un *chemin* d'attention sur 2 couches (direct, via P_1 , via P_2) : composer les couches capture les flux **indirects**.

Pourquoi $\frac{1}{2}(\mathbf{A} + \mathbf{I})$?

La **connexion résiduelle** : un token transmet aussi sa propre valeur à la couche suivante. Sans le $+\mathbf{I}$, on oublierait ce canal direct et le produit sous-estimerait les patches vus tôt.

Lire une explication — repérer les « raccourcis »



✓ Explication saine

La carte est **sur l'animal** → le modèle utilise le bon indice.



⚠ Signal d'alarme

La carte est **sur le fond** (neige) → *raccourci* : le modèle décide via le décor.

💡 Pourquoi c'est crucial

Exemple célèbre : un classifieur « husky vs loup » qui regardait en réalité la **neige** de l'arrière-plan (Ribeiro et al., 2016). L'explication révèle un **raccourci** (biais du dataset) totalement invisible sur la seule *accuracy*.

Réflexe d'audit

Comparer la carte aux **annotations expertes** ; croiser **plusieurs méthodes**. Si l'explication pointe le fond ou un artefact (étiquette d'hôpital, watermark) ⇒ corriger les *données*, pas seulement le modèle.

Comparaison des méthodes XAI

Méthode	Type d'attribution	Modèle requis	Données ciblées	Coût	Cas d'usage
Saliency	Pixel ($\nabla_x f$)	Différentiable	Image, signal	Très faible	Diagnostic rapide
Grad-CAM	Spatiale grossière (CNN)	Convolutif	Image	Faible	Médical, défauts
Integrated Grad.	Pixel, axiomatique	Différentiable	Image, tabulaire	Moyen	Audit modèle profond
SHAP	Feature (additif global)	Boîte noire ou tree	Tabulaire, image	Élevé	Conformité, finance
LIME	Superpixel ou feature	Boîte noire	Image, texte	Élevé	Métiers, présentation
Attention Rollout	Patch (ViT)	Transformer	Image, texte	Faible	Interprétabilité ViT

💡 Choix selon la situation

- **Vision CNN** : Grad-CAM en première intention ; Integrated Gradients pour audit
- **Vision ViT** : Attention Rollout ou Transformer Attribution
- **Tabulaire** : SHAP (TreeSHAP si XGBoost / DeepSHAP si DL)
- **Boîte noire** (API externe) : LIME ou KernelSHAP

Écosystème PyTorch pour la XAI

Bibliothèques officielles

- **Captum** (Facebook AI, intégré à PyTorch) : [IntegratedGradients](#), [DeepLift](#), [GradientShap](#), [Occlusion](#), [Saliency](#), [LayerGradCam](#)
- **pytorch-grad-cam** : Grad-CAM, HiResCAM, EigenCAM, FullGrad pour CNN et ViT
- **shap** : DeepExplainer, GradientExplainer, KernelExplainer
- **lime** : LimeImageExplainer, LimeTabularExplainer

Bonnes pratiques d'audit

Toujours combiner **plusieurs méthodes** ; comparer aux annotations expertes ; vérifier que l'explication ne dépend pas d'artefacts (ex. étiquettes de l'hôpital sur les radios).

```
from captum.attr import IntegratedGradients
from captum.attr import LayerGradCam
```

```
# Integrated Gradients
ig = IntegratedGradients(model)
attr_ig, delta = ig.attribute(
    inputs=x,
    target=target_class,
    baselines=torch.zeros_like(x),
    n_steps=50,
    return_convergence_delta=True)
```

```
# Grad-CAM via Captum
gc = LayerGradCam(model, model.layer4[-1])
attr_gc = gc.attribute(
    inputs=x, target=target_class)
```

```
# pytorch-grad-cam (alternative)
from pytorch_grad_cam import GradCAM
cam = GradCAM(model=model,
               target_layers=[model.layer4[-1]])
grayscale_cam = cam(input_tensor=x)
```

Synthèse & références

Synthèse mathématique du Module 2

Concept	Formule clé	PyTorch / outil
Transfer feature extr.	Gel : <code>requires_grad=False</code> ; remplacement de la tête	<code>torchvision.models.resnet50</code>
LR différentiel	$\{\eta_{\text{backbone}}=10^{-5}, \eta_{\text{head}}=10^{-3}\}$	<code>optim.Adam([{\dots}])</code>
OneCycleLR	Warm-up linéaire $\rightarrow$ annealing cosinus	<code>optim.lr_scheduler.OneCycleLR</code>
RNN forward	$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$	<code>nn.RNN</code>
LSTM cell update	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	<code>nn.LSTM</code>
GRU update	$h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	<code>nn.GRU</code>
Gradient clipping	$g \leftarrow g \cdot \min(1, \tau/\ g\)$	<code>clip_grad_norm_</code>
Saliency	$S_i = \partial f_c / \partial x_i $ (1 backward)	<code>captum.Saliency</code>
Grad-CAM	$L_c = \text{ReLU}(\sum_k \alpha_c^k A^k)$	<code>captum.LayerGradCam</code>
Integrated Gradients	$(x_i - x'_i) \int_0^1 \partial F / \partial x_i d\alpha$	<code>captum.IntegratedGradients</code>
SHAP	$\phi_i = \sum_S w_S [v(S \cup \{i\}) - v(S)]$	<code>shap.DeepExplainer</code>
LIME	$\arg \min_g \mathcal{L}(f, g, \pi_x) + \Omega(g)$	<code>lime.LimeImageExplainer</code>
Attention Rollout (ViT)	$R = \prod_i \frac{1}{2} (A^{(l)} + I)$	implémentation custom

Trois ouvrages couvrent les concepts présentés

1. **Atienza, R.** (2018). *Advanced Deep Learning with Keras*. Packt Publishing. ISBN 978-1-78862-941-6.
Chapitres pertinents : RNN/LSTM, transfer learning, autoencoders.
2. **Vasilev, I.** (2019). *Python Deep Learning* (2^e éd.). Packt Publishing.
Chapitres pertinents : modèles récurrents, fine-tuning, attention.
3. **Buduma, N., Buduma, N., & Papa, J.** (2022). *Fundamentals of Deep Learning* (2^e éd.). O'Reilly Media. ISBN 978-1-492-08128-7.
Chapitres pertinents : interprétabilité (Grad-CAM, IG), séquences, optimisation.

Les références couvrent les fondements théoriques de toutes les architectures vues dans la formation (Modules 1 à 4).

Du transfer à l'explication.

Cap sur le Module 3 — Autoencoders, VAE & Diffusion

 bassem.benhamed@enetcom.usf.tn